

虚拟内存-02

古金宇

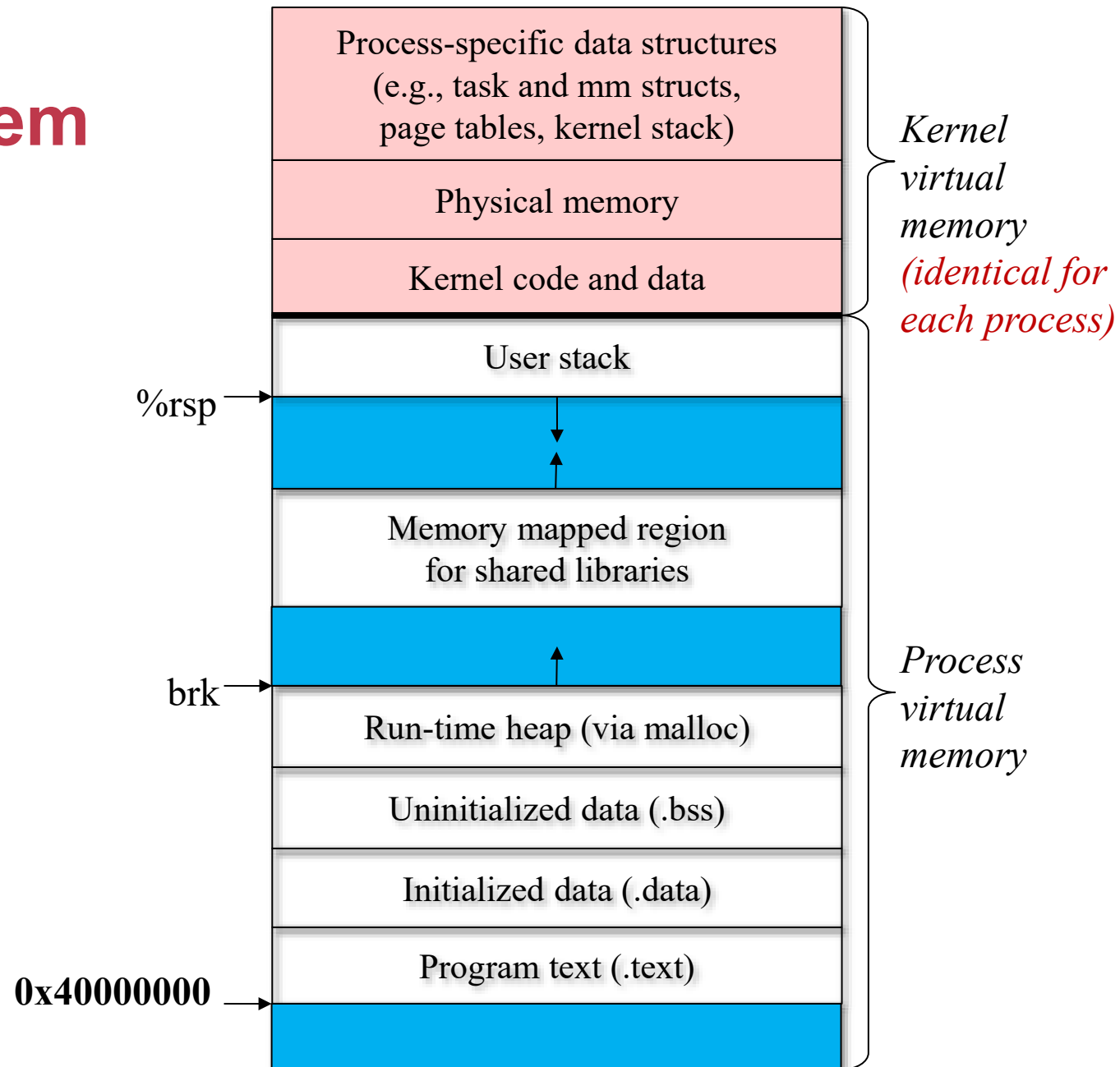
上海交通大学并行与分布式系统研究所

<https://ipads.se.sjtu.edu.cn>

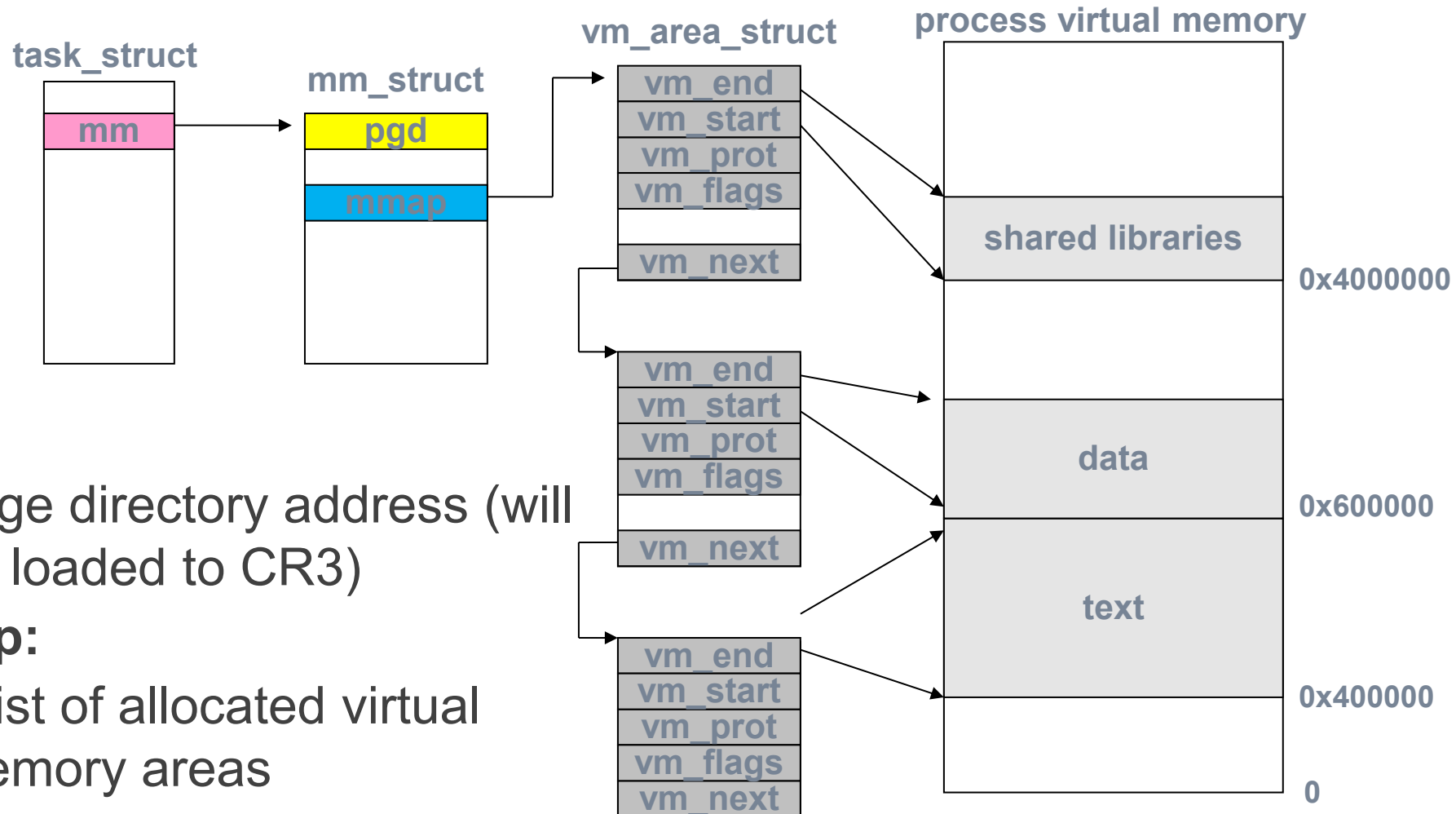
Outline

- Memory Mapping
- Suggested reading: 9.7.2, 9.8
- Replacement Policy
- Suggested reading: OSTEP 22, 银杏书（物理内存章节）

Linux Virtual Memory System

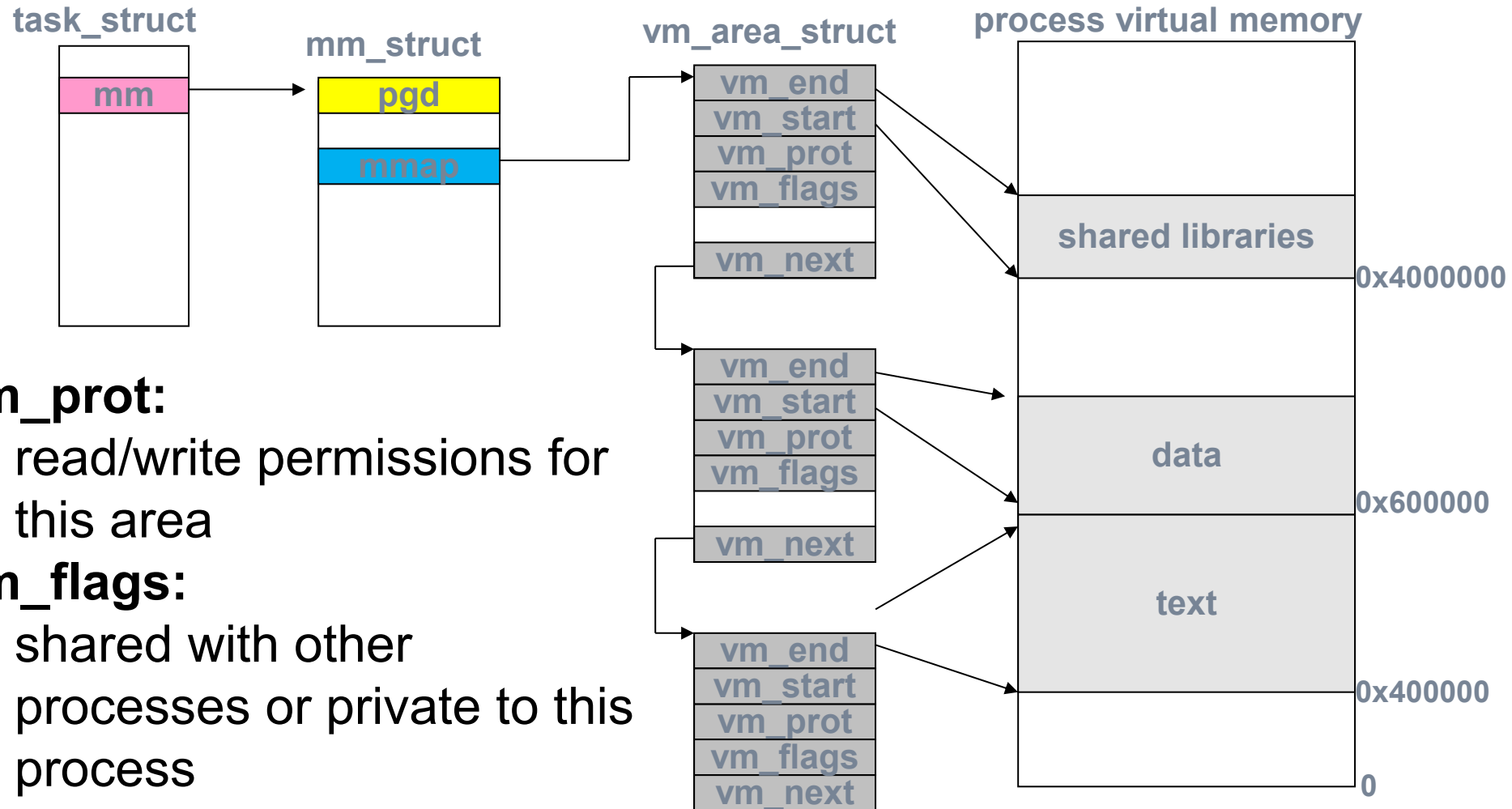


Linux organizes VM as a collection of “areas”



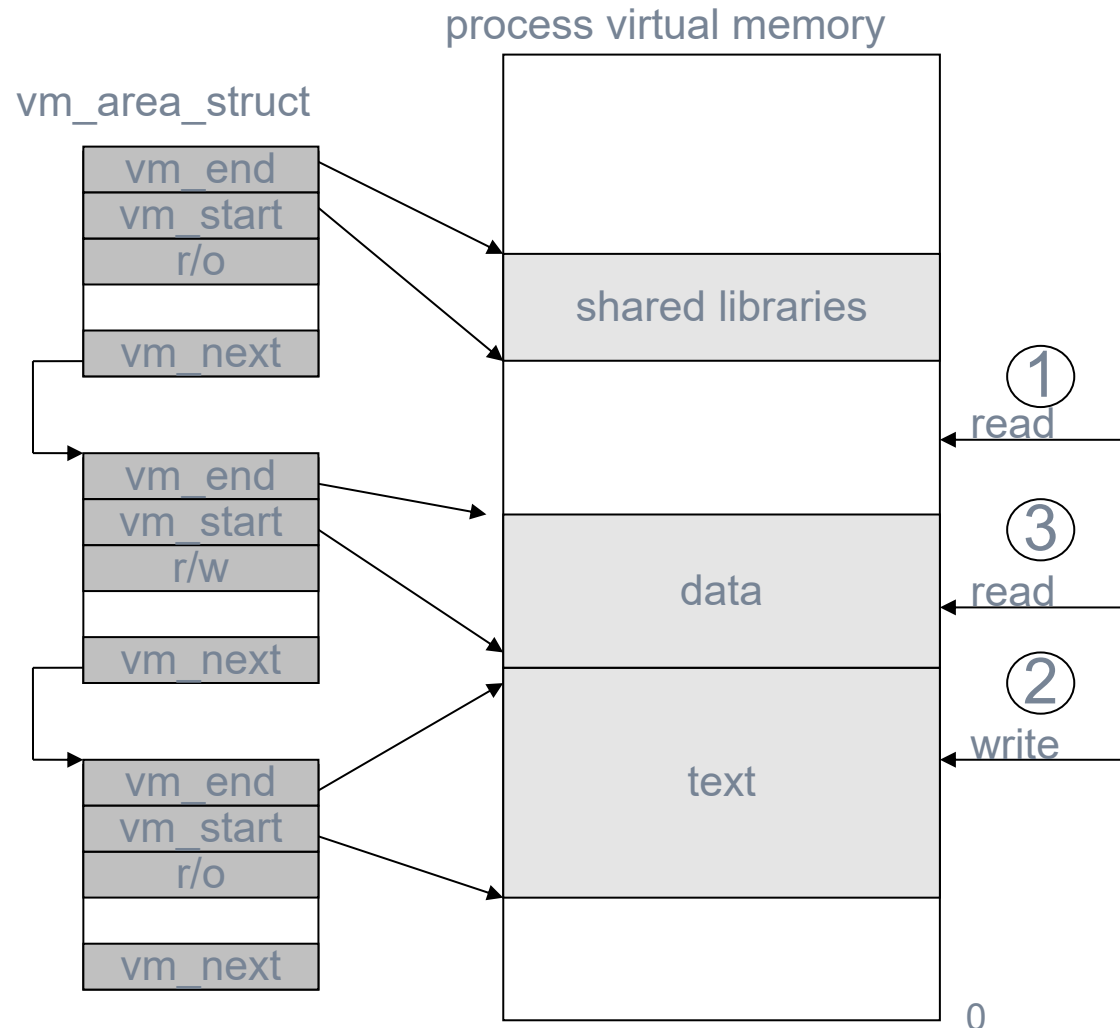
- **pgd:**
 - page directory address (will be loaded to CR3)
- **mmap:**
 - a list of allocated virtual memory areas

Linux organizes VM as a collection of “areas”



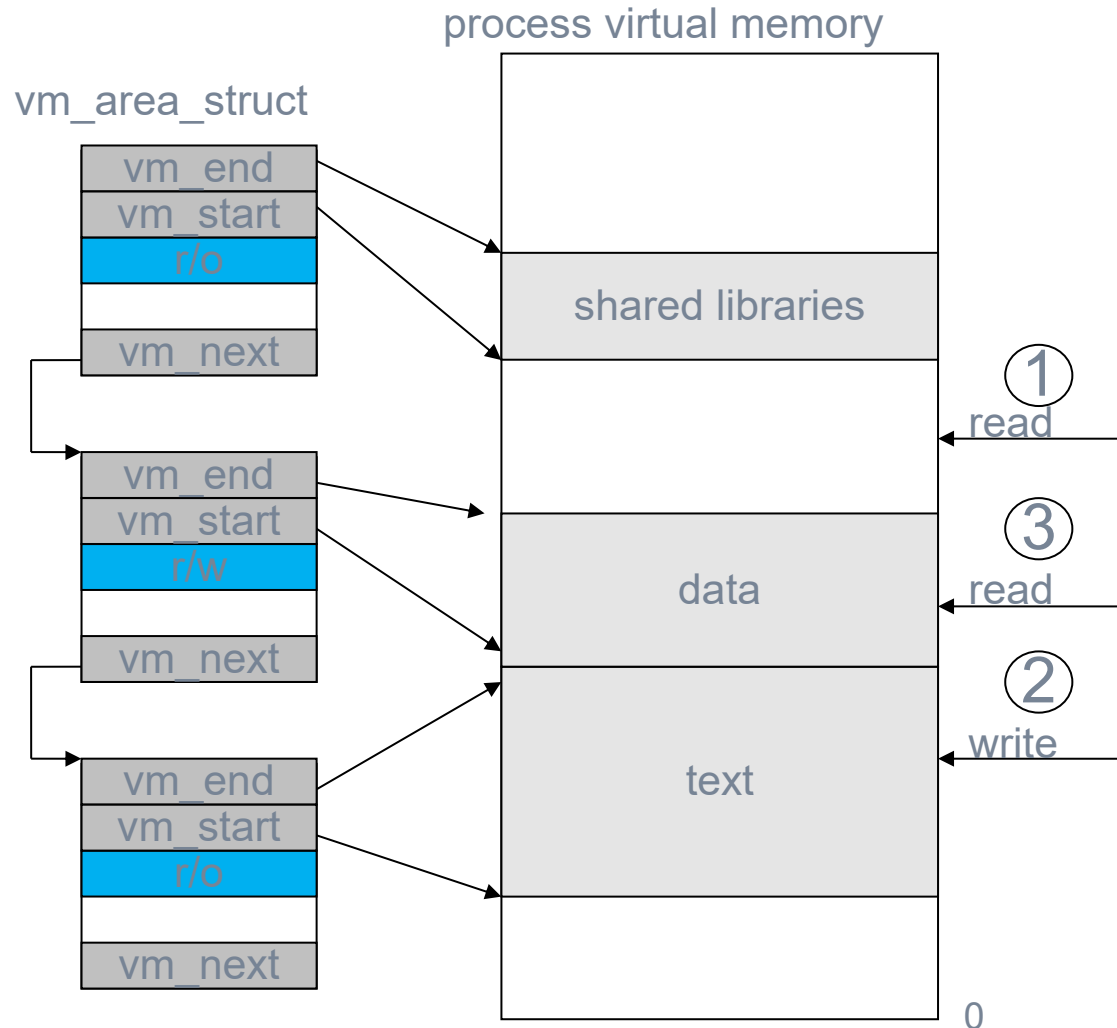
- **vm_prot:**
 - read/write permissions for this area
- **vm_flags:**
 - shared with other processes or private to this process

Linux page fault handling



- Is the VA legal?
 - i.e., is it in an area defined by a **vm_area_struct**?
 - if not then signal segmentation fault (e.g., ①)

Linux page fault handling



- Is the operation legal?
 - i.e., can the process read/write this area?
 - if not then signal protection violation (e.g., ②)
- If OK, handle fault
 - e.g., ③

Memory mapping

- Memory mapping
 - Linux initializes the contents of a VM area by associating it with a disk object
 - A VM area can get its initial values from the object
- Area can be backed by
 - regular file on disk (e.g., an executable object file)
 - initial page bytes come from a section of a file
 - anonymous file (nothing)
 - First fault will allocate a physical page full of 0's (demand-zero page)
 - Once the page is written to (dirtied), it is like any other page

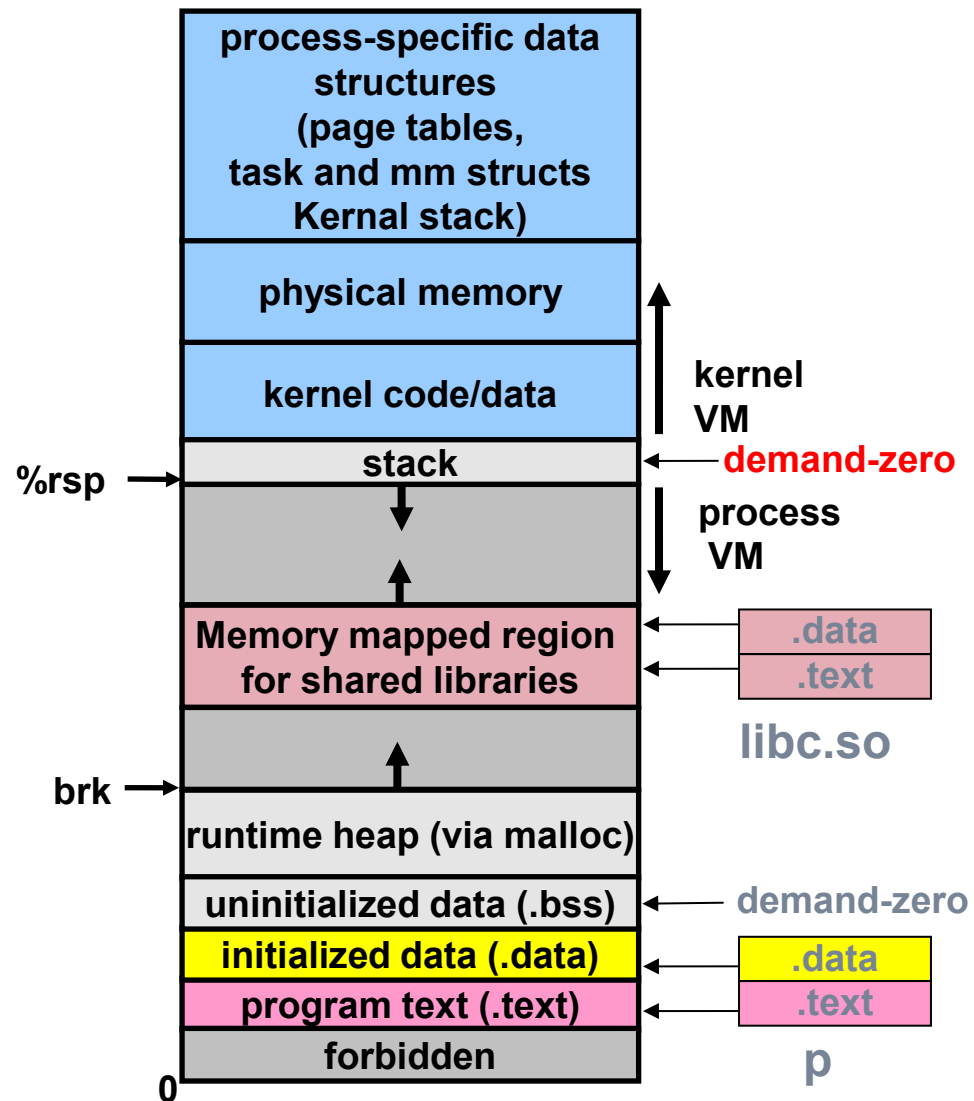
Memory swapping

- Swap in/out
 - Dirty pages can be copied back and forth between memory and a **swap file**
- Swap file
 - Maintained by the kernel
 - Also known as swap space or the swap area

Demand Paging

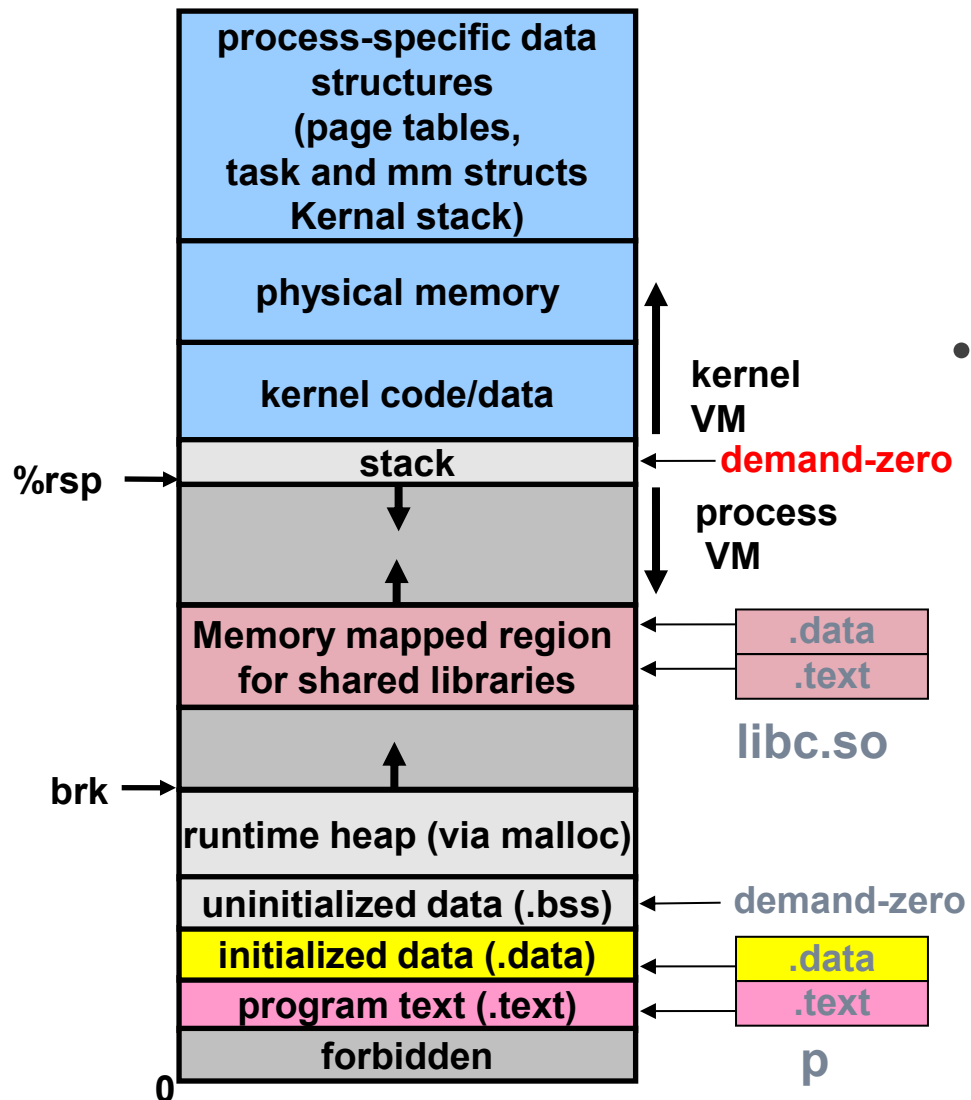
- Key point: no virtual pages are copied into physical memory until they are referenced!
 - known as “demand paging”
 - crucial for time and space efficiency

Exec() revisited



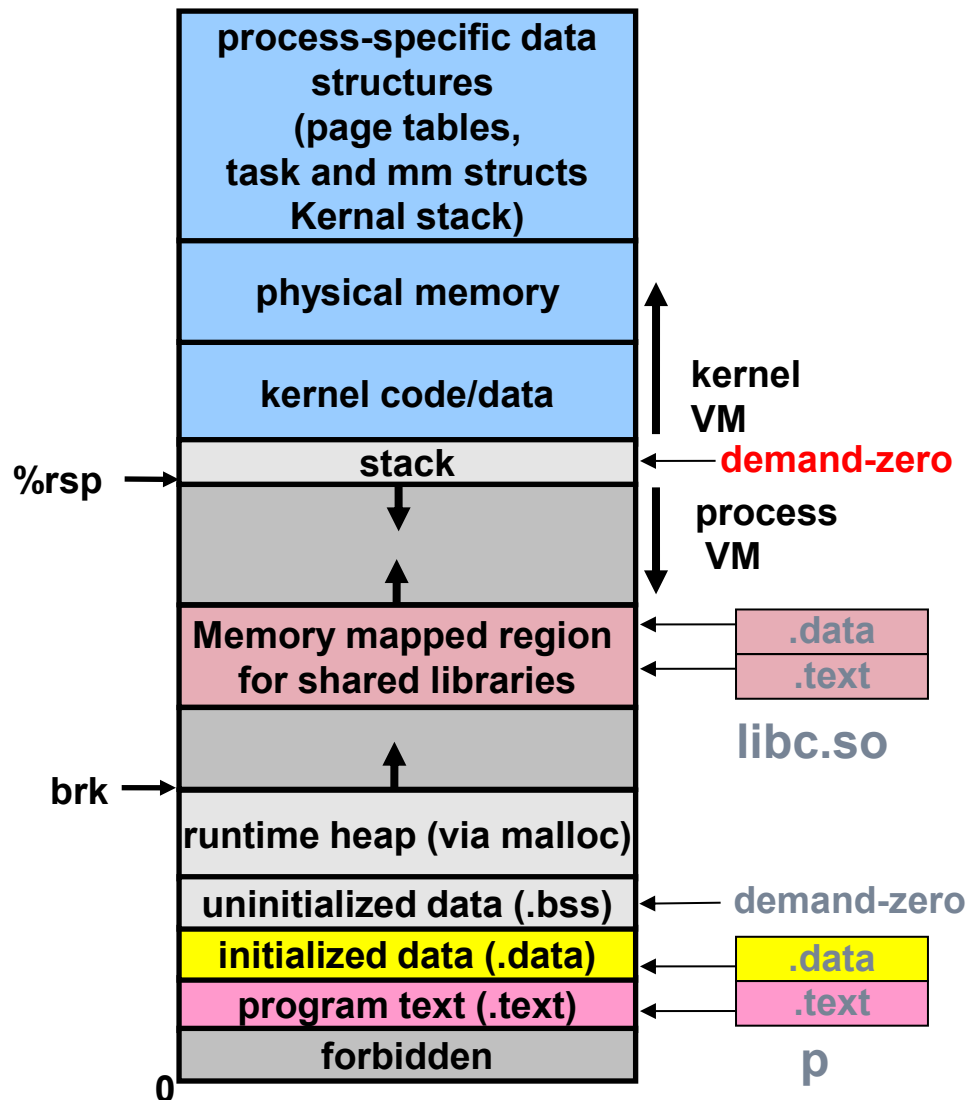
- To run a new program `p` in the current process using `exec()`:
 - free all `vm_area_structs` and `page tables` for old areas

Exec() revisited



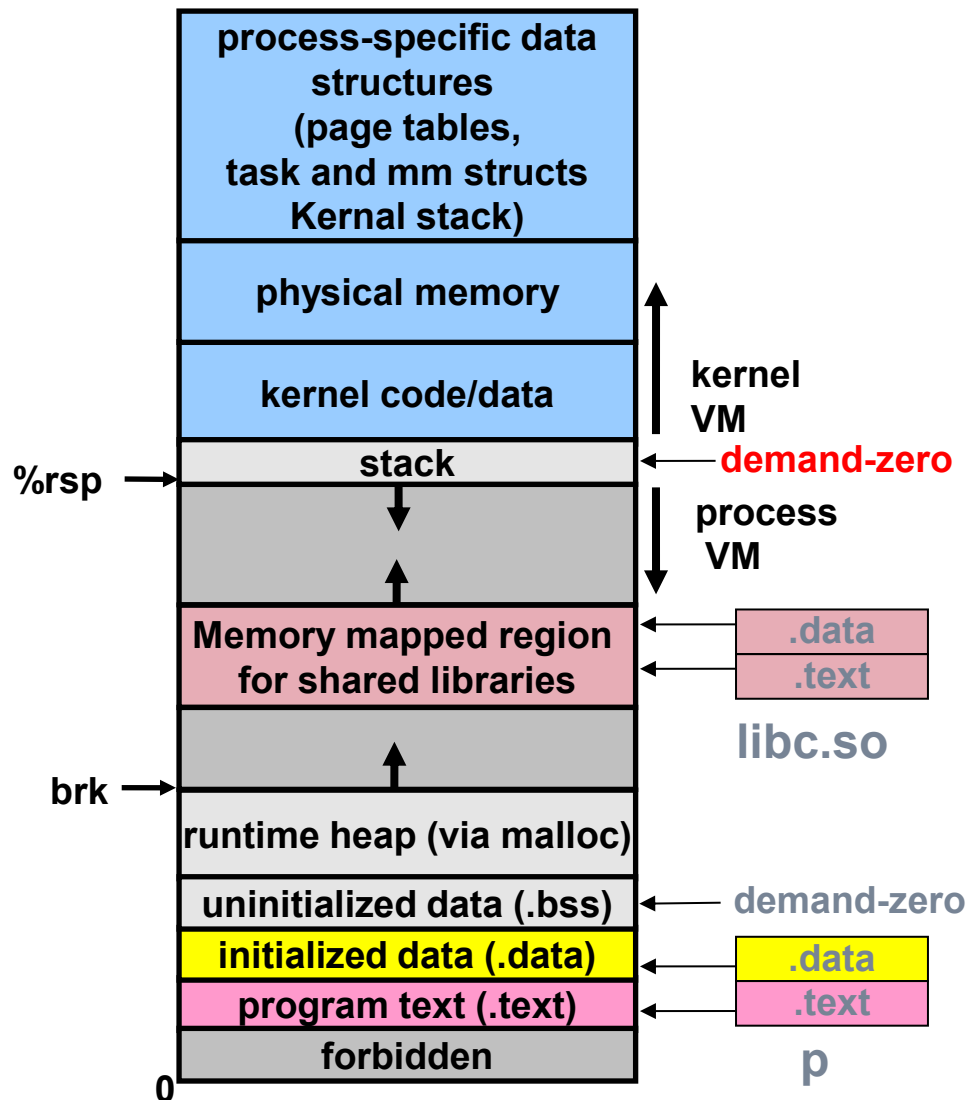
- To run a new program `p` in the current process using `exec()`:
 - create new `vm_area_structs` and `page tables` for new areas:
stack, bss, data, text, shared libs

Exec() revisited



- To run a new program `p` in the current process using `exec()`:
 - create new `vm_area_structs` and page tables for new areas:
 - **text** and **data** backed by ELF executable object file
 - **bss** and **stack** initialized to zero

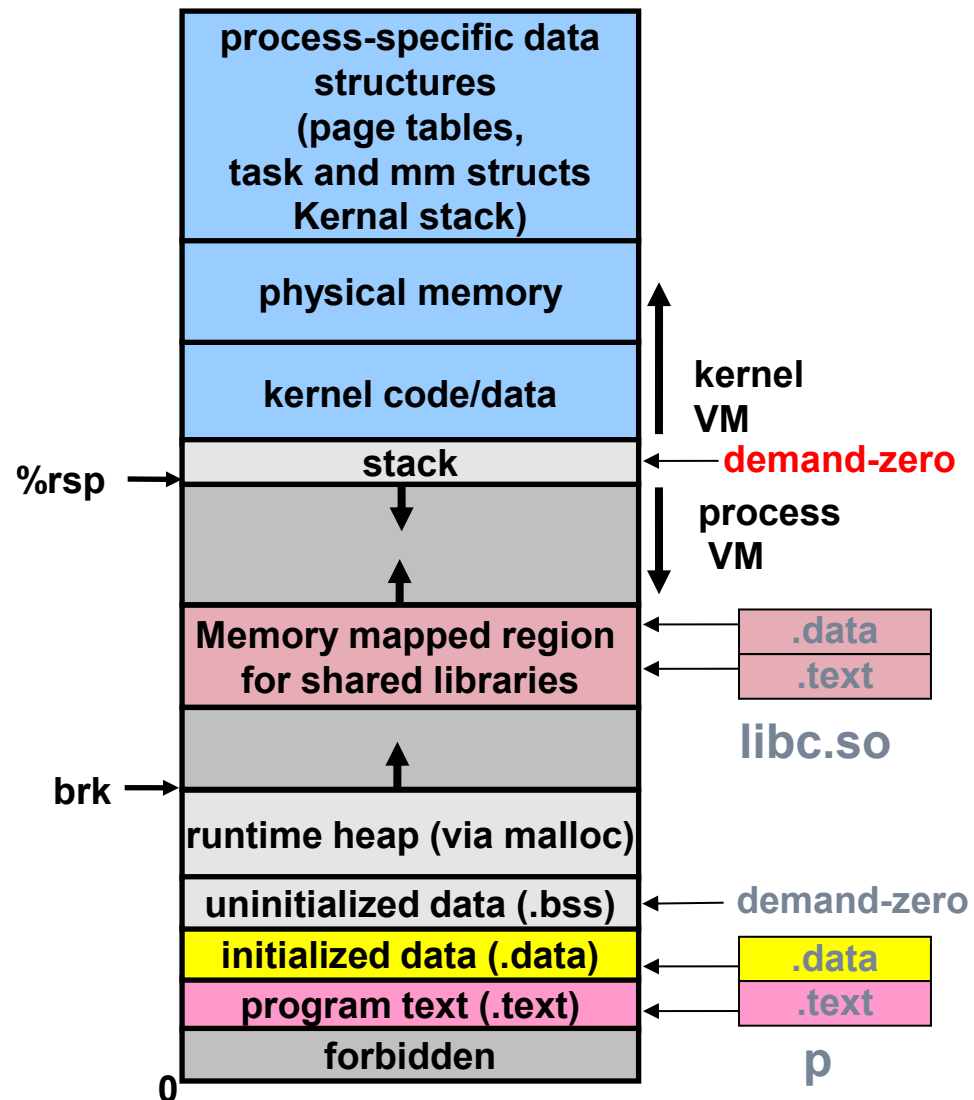
Exec() revisited



- To run a new program `p` in the current process using `exec()`:
 - create new `vm_area_structs` and page tables for new areas:
 - `text`, `data`, `bss`, and `stack` area are private areas
 - Q: `Shared libs`: private or shared areas?

Private areas!

Exec() revisited



- To run a new program `p` in the current process using `exec()`:
 - set PC to entry point in `.text`
 - Linux will do paging for code and data pages **on demand**

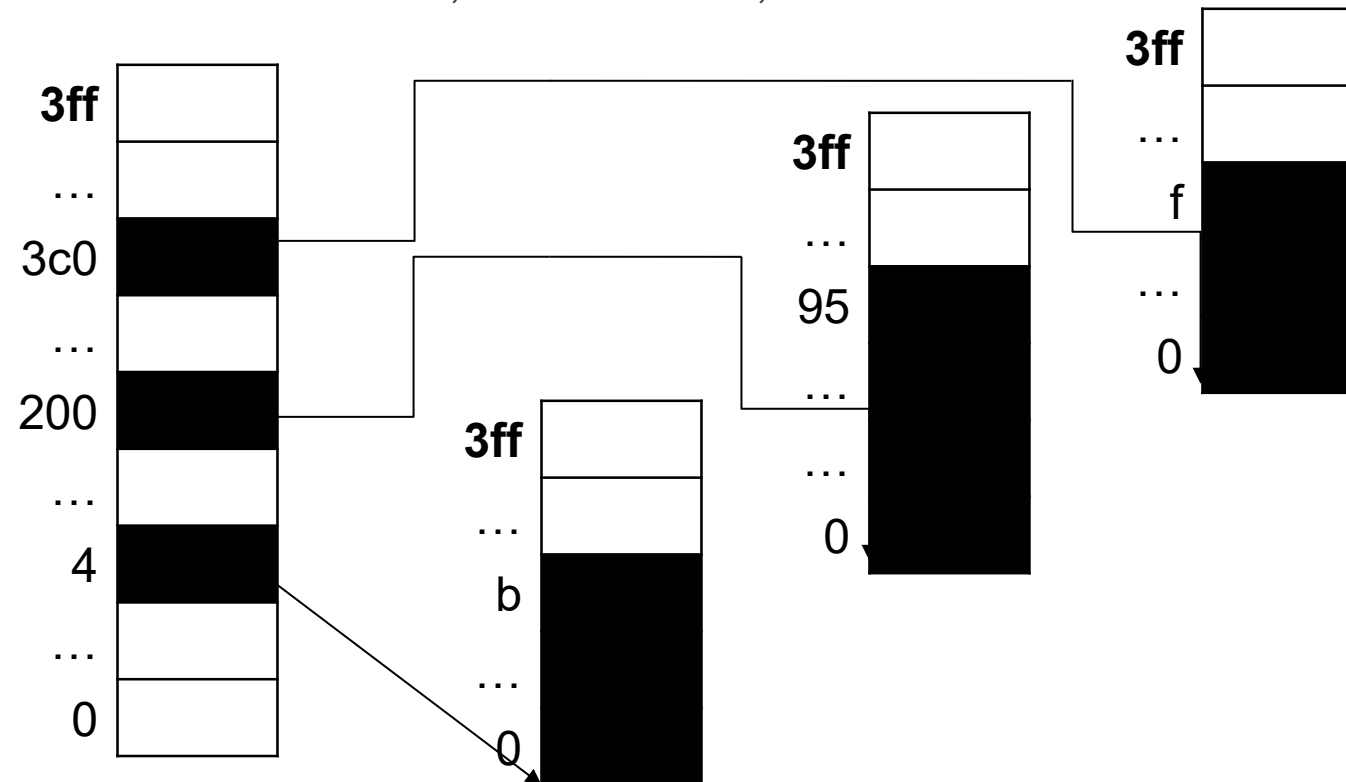
Example

- A 32-bit machine with 2-level page table of 10-10-12 structure
- What is the skeleton of the page table for a process that has
 - a code segment of 48K starting at address 0x1000000,
 - a data segment of 600K starting at address 0x80000000
 - and a stack segment of 64K starting at address 0xf0000000 and growing upward (towards higher addresses)?

Example

1 000 000	00 0000 0100	00 0000 0000	0000 0000 0000
80 000 000	10 0000 0000	00 0000 0000	0000 0000 0000
f0 000 000	11 1100 0000	00 0000 0000	0000 0000 0000

48K=4K*12, 600K=4K*150, 64K=4K*16



User-level memory mapping

```
void *mmap(void *start, int len, int prot,  
           int flags, int fd, int offset)
```

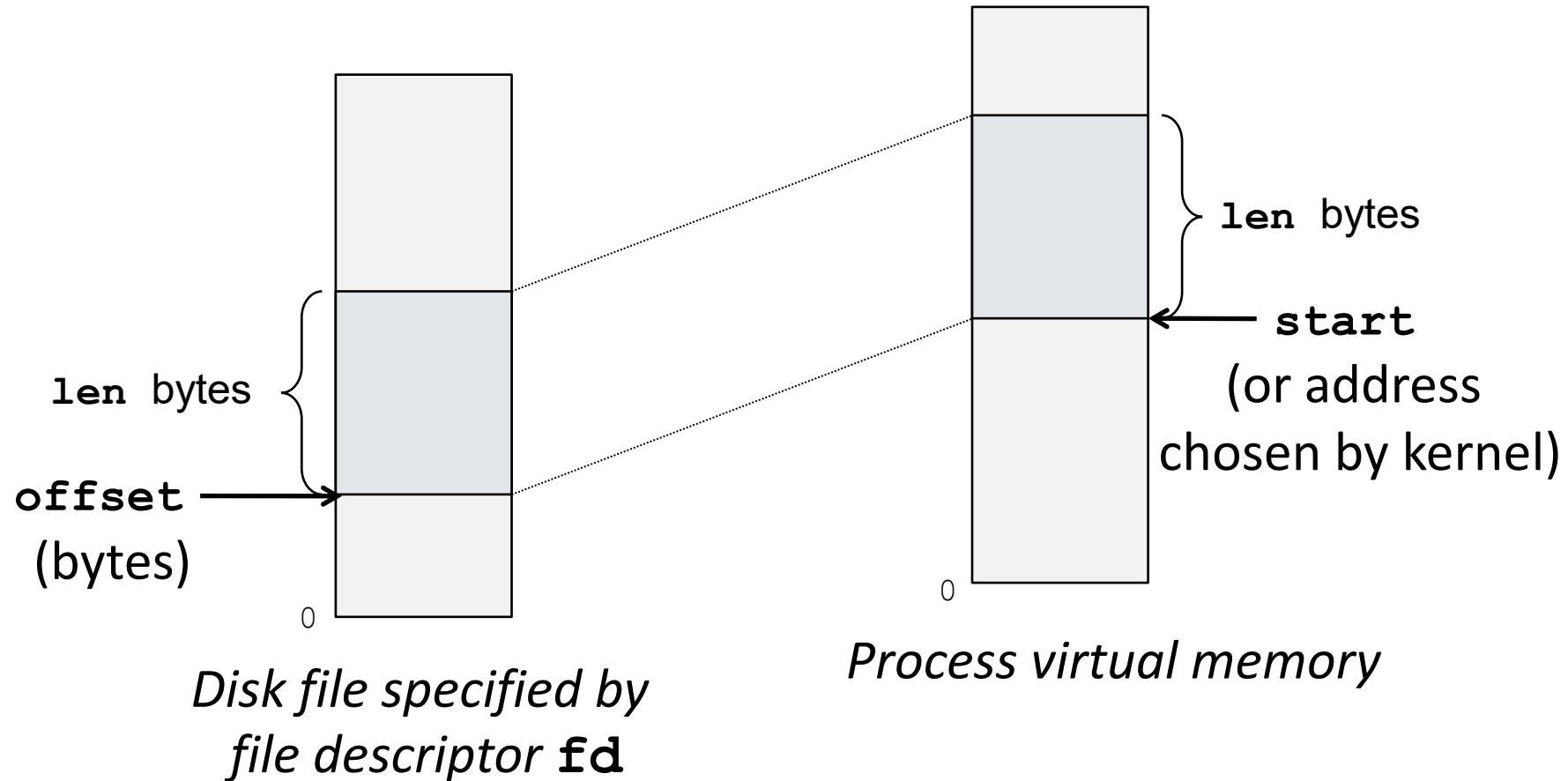
Map *len* bytes starting at offset *offset* of the file specified by file description *fd*, preferably at address *start* (usually NULL for don't care).

- *prot*: MAP_READ, MAP_WRITE
- *flags*: MAP_PRIVATE, MAP_SHARED

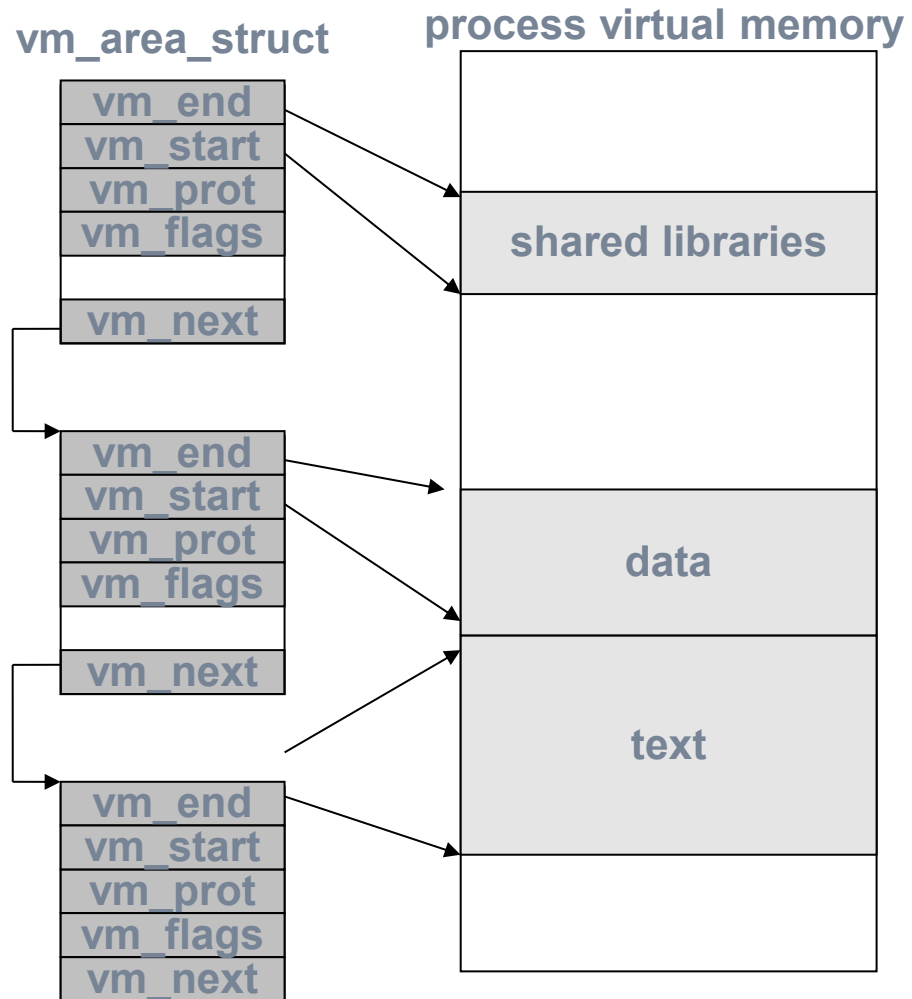
Return a pointer to the mapped area

User-level memory mapping

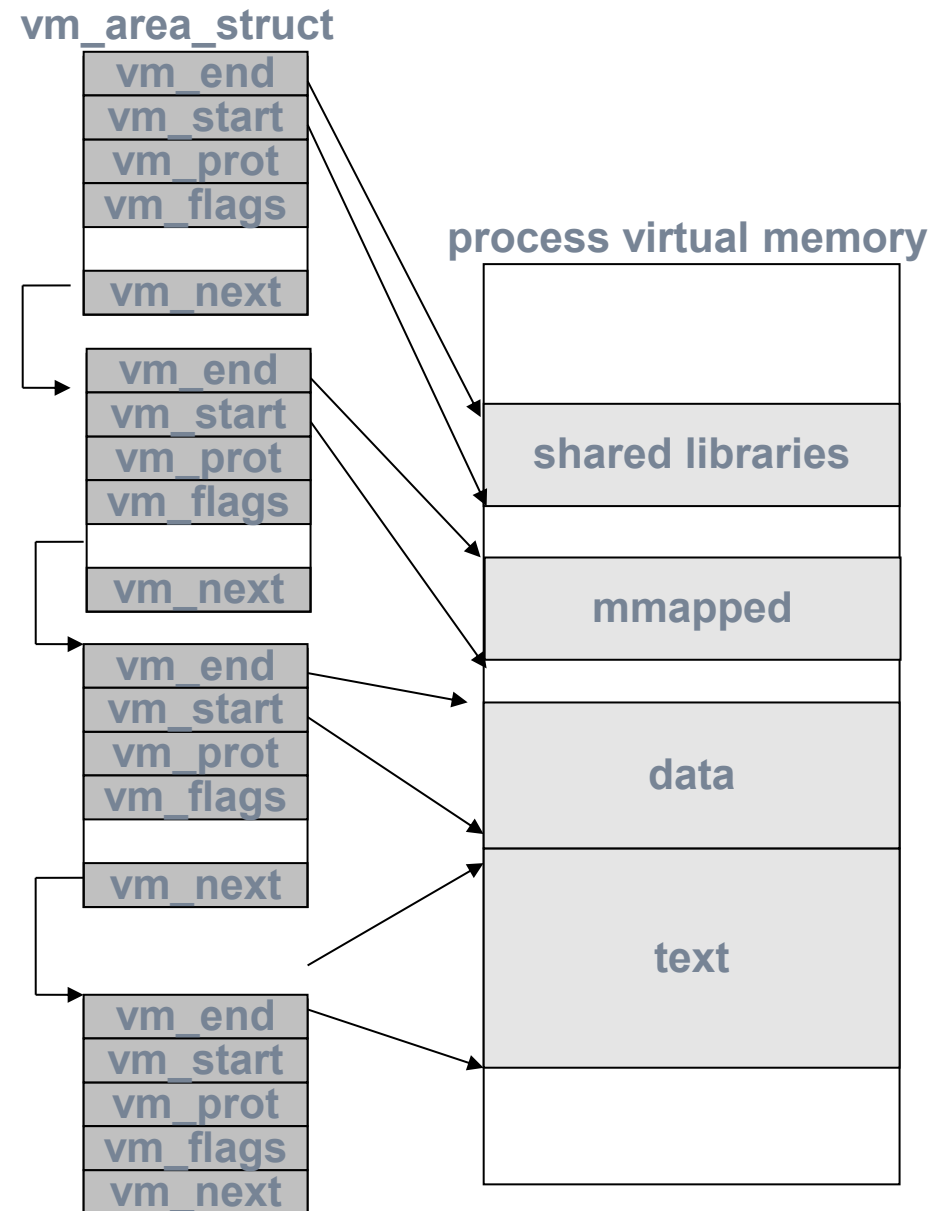
```
void *mmap(void *start, int len,  
           int prot, int flags, int fd, int offset)
```



VMA changes



Before mmap



After mmap

User-level memory mapping

- Example: fast file copy
 - Useful for applications like Web servers that need to quickly copy files
 - **mmap** allows file transfers without copying into user space

mmap() example: fast file copy

```
/*
 * mmapcopy - uses mmap to copy file fd to stdout
 */
void mmapcopy(int fd, int size)
{
    char *bufp;

    /* map the file to a new VM area */
    bufp = mmap(0, size, PROT_READ,
                MAP_PRIVATE, fd, 0);

    /* write the VM area to stdout */
    write(1, bufp, size);

    return ;
}
```

mmap() example: fast file copy

```
int main(int argc, char **argv)
{
    struct stat stat;
    /* check for required command line argument */
    if ( argc != 2 ) {
        printf("usage: %s <filename>\n", argv[0]);
        exit(0) ;
    }
    /* open the file and get its size*/
    fd = open(argv[1], O_RDONLY, 0);
    fstat(fd, &stat);
    mmapcopy(fd, stat.st_size);
}
```

Q: Why is this faster?

Example

- 下列程序运行在32位Linux系统上

```
#define PAGE_SIZE 4 * 1024
```

```
int main(void) {
```

```
    char *p = NULL;
```

```
    int i;
```

```
    A:    p = mmap(0, 128 * PAGE_SIZE, PROT_READ |  
                PROT_WRITE, MAP_PRIVATE | MAP_ANON, -1, 0);
```

```
    printf("buffer start: %p\n", p);
```

```
    for(i = 0; i < BUF_SIZE; i += PAGE_SIZE)
```

```
        p[i] = 1;
```

```
    B:    munmap(p, BUF_SIZE);
```

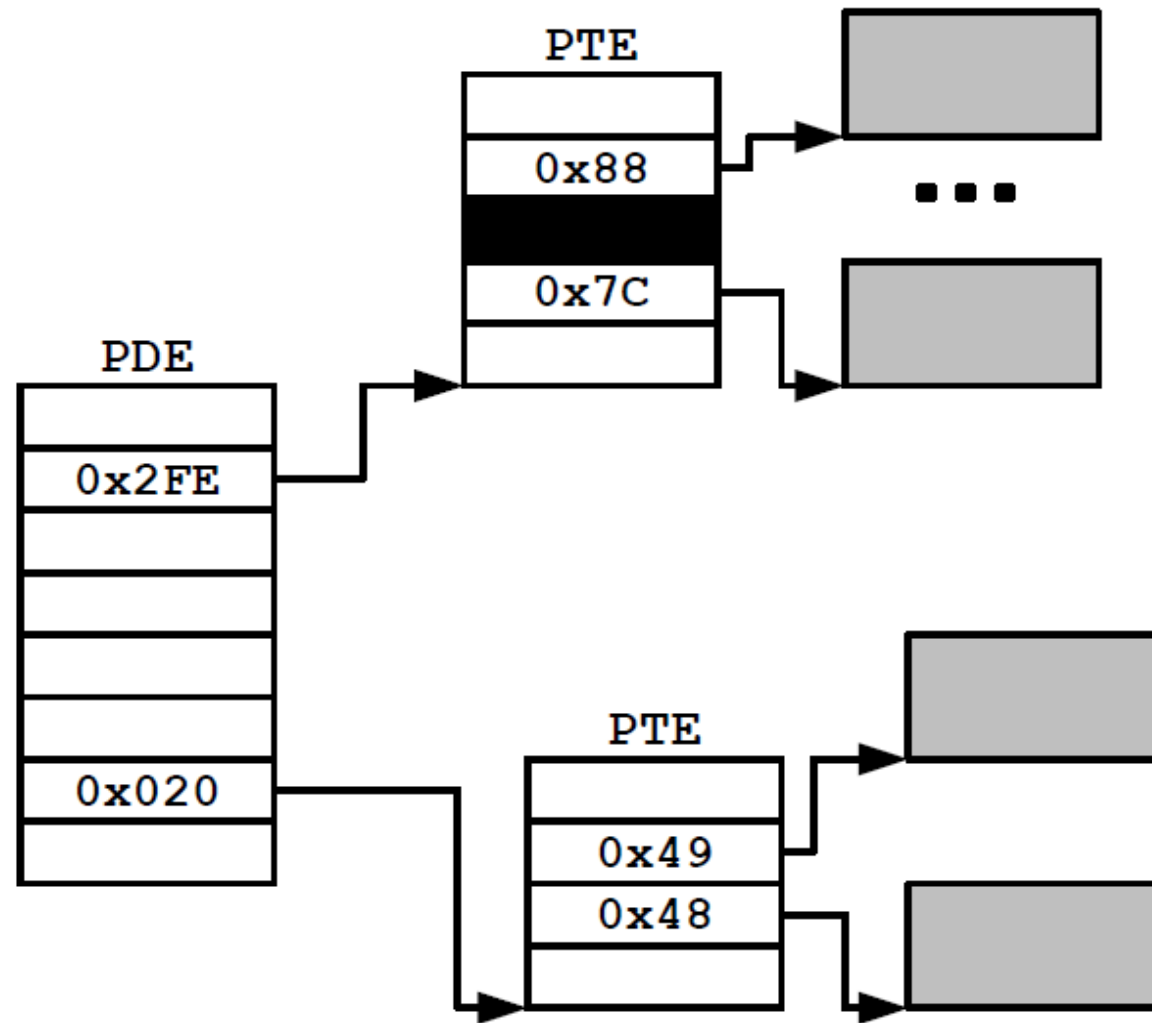
```
    return;
```

```
}
```

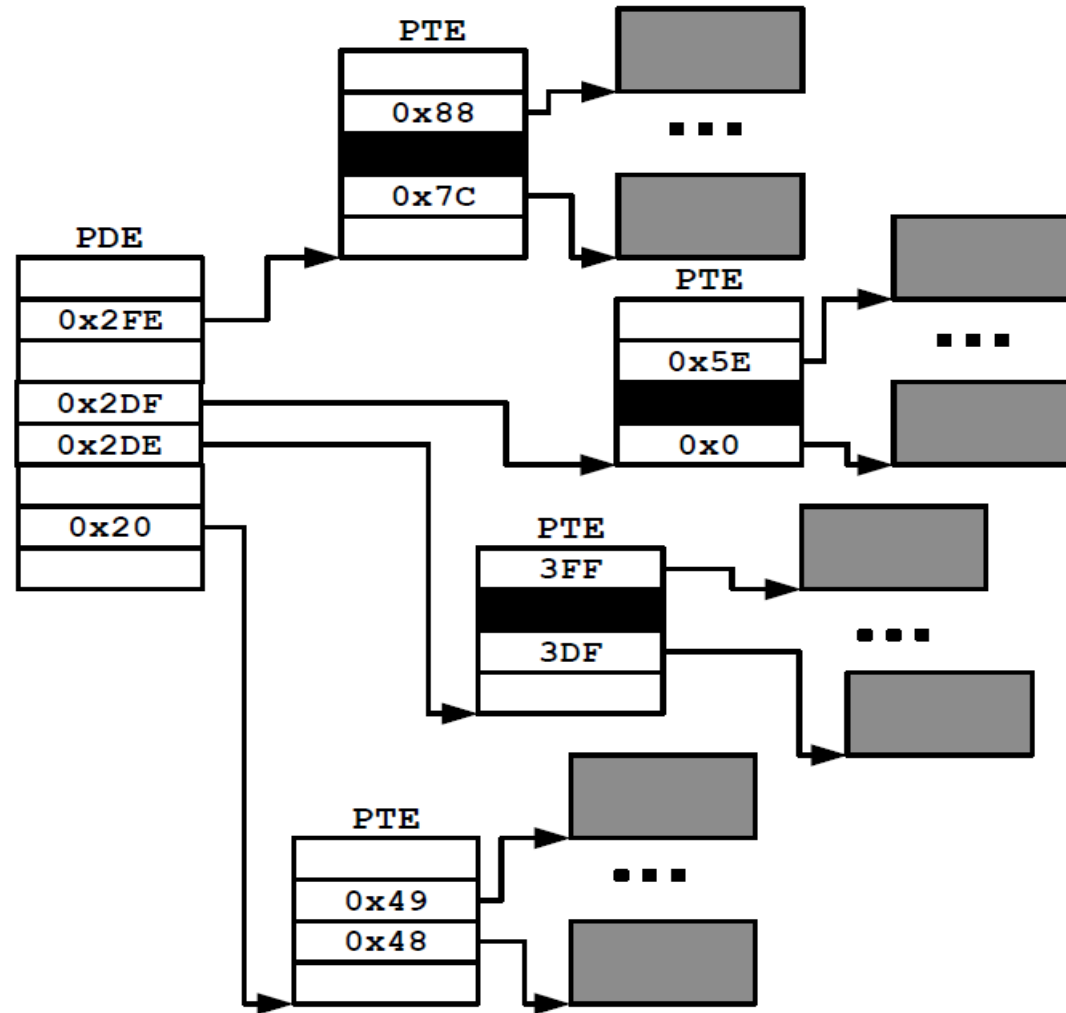
Example

- 当程序到达标号A时，页表如下。块中的数字表示PDE/PTE的偏移值。函数printf的输出值是 “**buffer start: 0xb7bdf000**” 。
- 请画出程序到达标号B时页表的形状。
 - **注意:** 没有数字的白色块意味着空的PDE/PTE，填写了数字的白色块表示PDE/PTE被使用，黑色块表示连续多个PDE/PTEs被使用，灰色块表示一个有数据/代码的页)

Example



A sample from exam papers



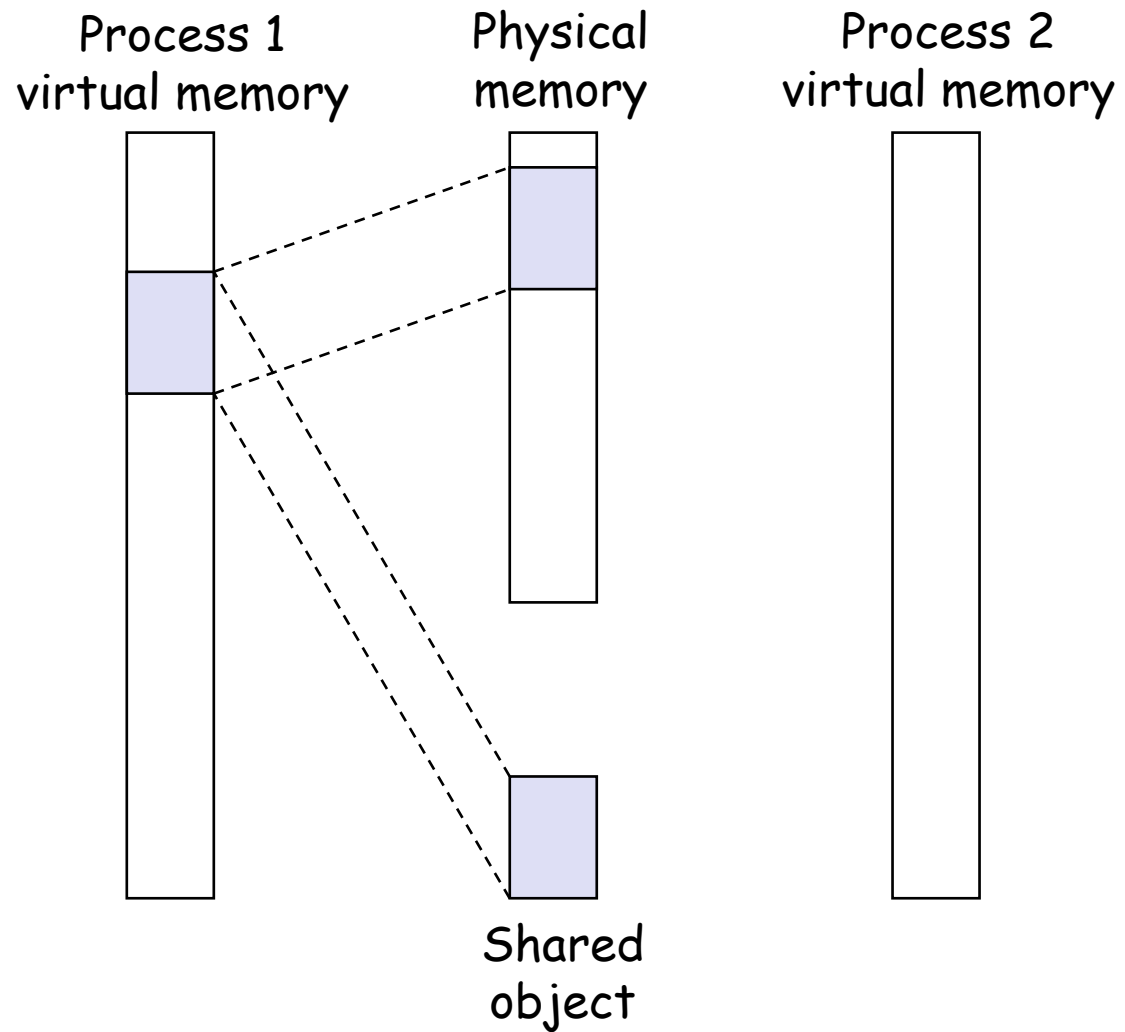
Fork() revisited

- To create a new process using **fork**:
 - Make copies of the old process's **mm_struct**, **vm_area_struct**, and **page tables**
 - Two processes are sharing all of their pages
(At this point)
 - How to get separate spaces without copying all the virtual pages from one space to another?
 - “**Copy On Write**” (COW) technique

Shared Object

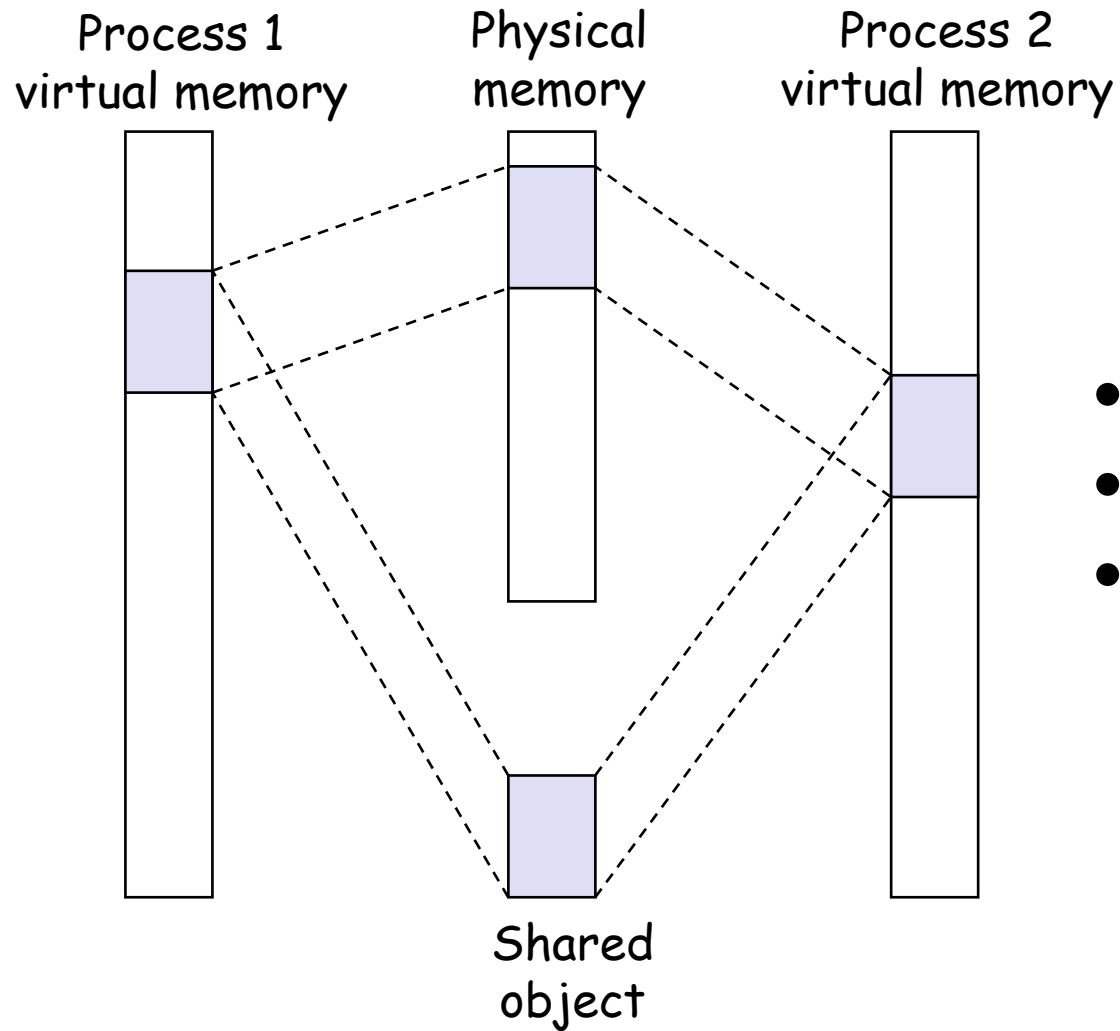
- Shared Object
 - An object which is mapped into an area of virtual memory of a process
 - Any writes that the process makes to that area are **visible to any other processes** that have also mapped the shared object into their virtual memory
 - The changes are also reflected in the original object on disk (**thus, shared flag is used when persistence is needed**)
- Shared Area
 - A virtual memory area that a shared object is mapped

Sharing a Shared Object



- **Process 1 maps the shared object**

Sharing a Shared Object



- Process 2 maps the shared object
- Notice the virtual addresses can be **different**
- Inter-process communication

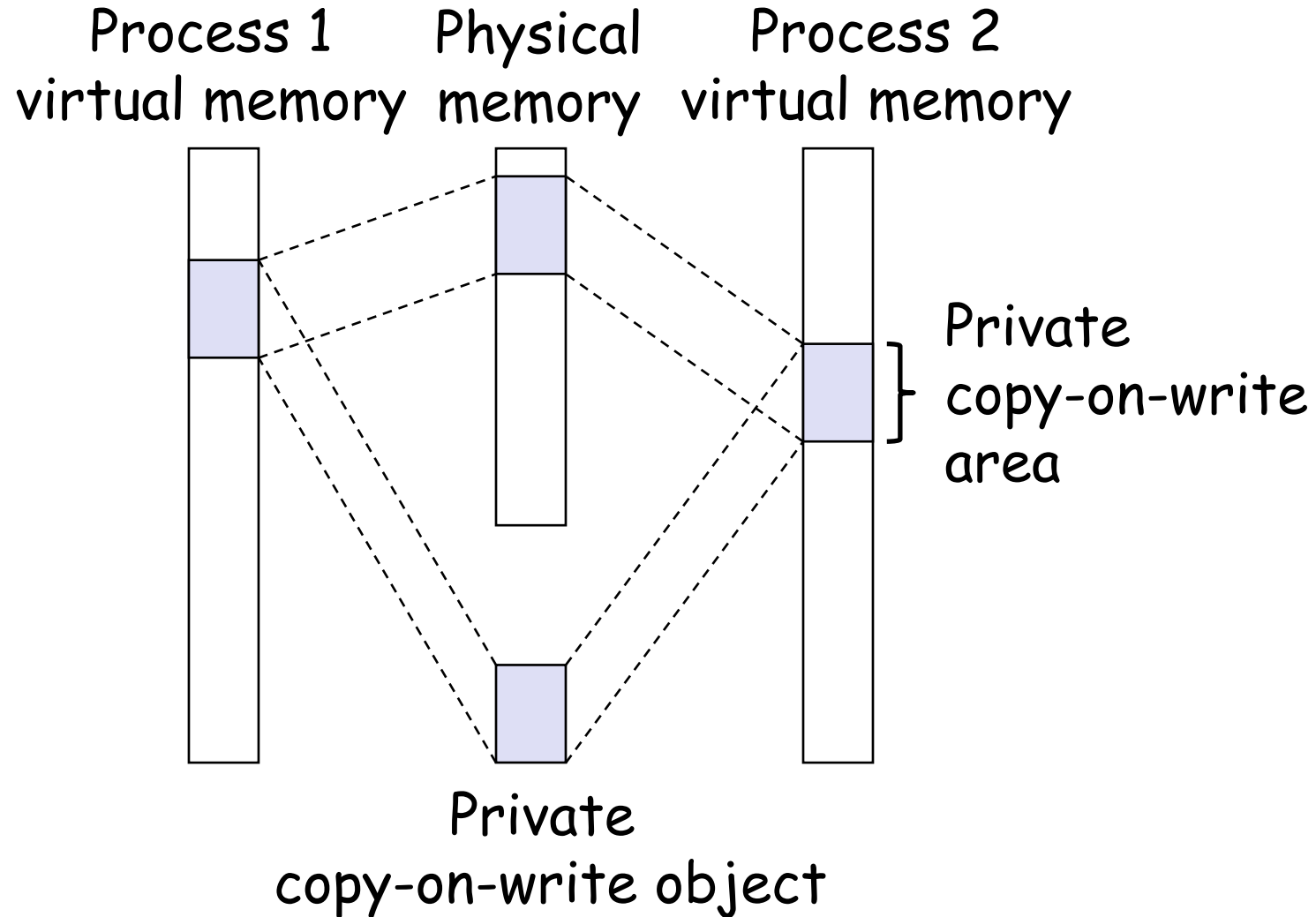
Private object

- Private Object
 - As oppose to shared object
 - Changes made to an area mapped to a private object are not visible to other processes
 - Any writes that the process makes to the area are not reflected back to the object on disk
- Private Area
 - Similar to shared area

Fork() revisited

- To create a new process using fork
 - Copy-On-Write
 - make pages of writeable areas **read-only**
 - flag **vm_area_struct** for these areas as **private** “copy-on-write”
 - writes by **either** process to these pages will cause **page faults**
 - fault handler recognizes copy-on-write, makes a copy of the page, and restores write permissions

Sharing a Private COW Object



Copy-on-Write

- For each process that maps the private object
 - The page table entries for the corresponding private area are flagged as **read-only**
 - The area struct is flagged as **private copy-on-write**
 - So long as neither process attempts to write to its respective private area, they continue to share a single copy of the object in physical memory

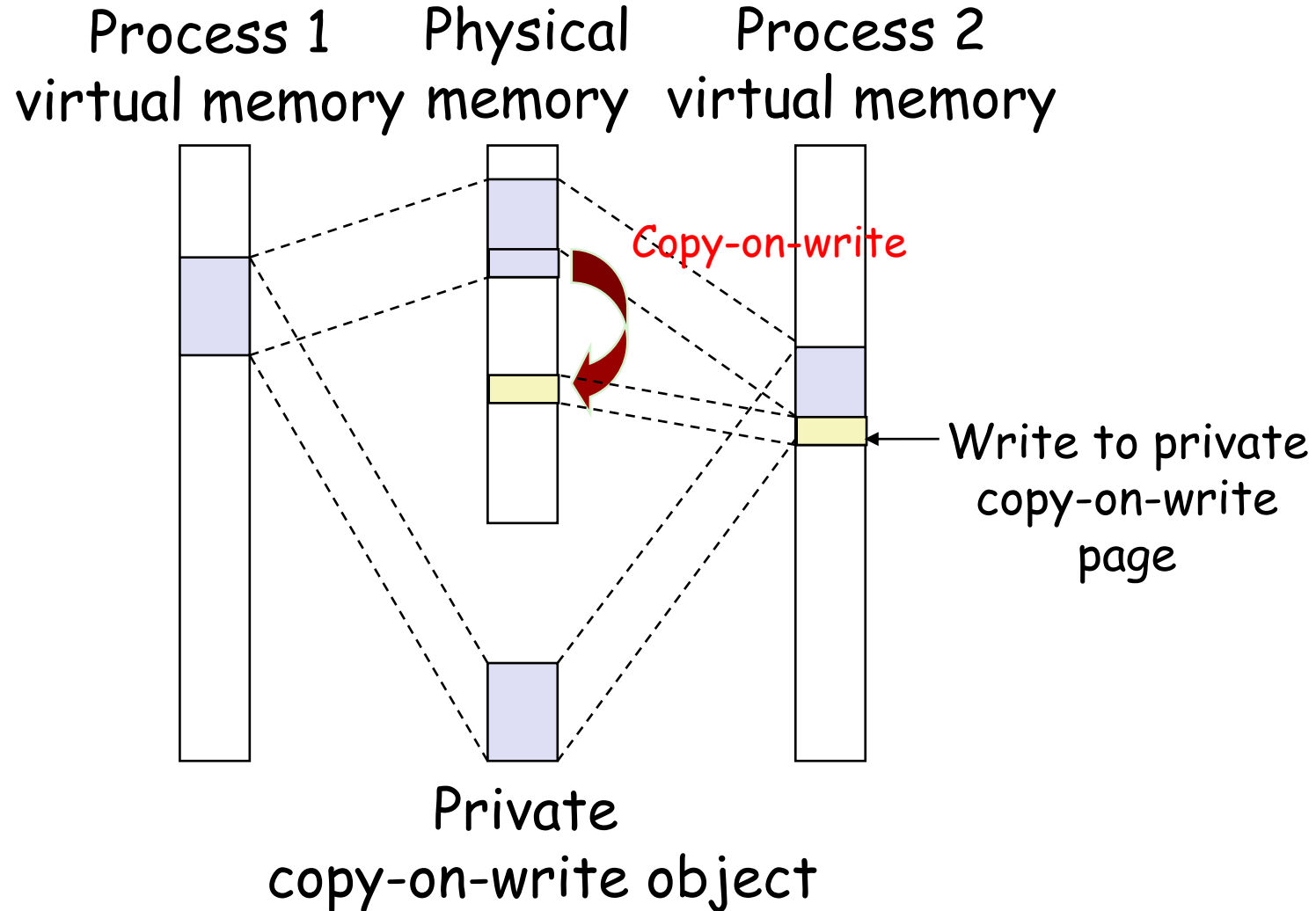
Copy-on-Write

- For each process that maps the private object
 - As soon as a process attempts to **write** to some page in the private area, the write triggers a protection fault
 - The fault handler notices that the protection exception was caused by the process trying to write to a page in a private copy-on-write area

Copy-on-Write

- For each process that maps the private object
 - The **fault handler**
 - Creates a new copy of the page in physical memory
 - Updates the page table entry to point to the new copy
 - Restores write permissions to the page

Sharing Revisited: Private COW Objects



Fork() revisited

- To create a new process using fork:
 - Net result:
 - copies are deferred until absolutely necessary
(i.e., when one of the processes tries to modify a shared page)

REPLACEMENT POLICY

页替换策略

When Page Swapping happens

- When the OS needs to allocate a free (physical) page but fails
- In-advance (Linux kswapd)
 - Set two numbers as low/high water mark
 - when available page number is less than the low water mark, OS begins to evict pages until the number of available pages reaches the high water mark

Memory Access Latency

- Terminologies
 - AMAT: Average memory access time
 - T_M : cost of accessing memory, around 100ns
 - T_D : cost of accessing disk, around 10ms
 - P_{hit} : hit rate
 - P_{miss} : miss rate

$$AMAT = (P_{Hit} \cdot T_M) + (P_{Miss} \cdot T_D)$$

- Miss rate means a lot
 - $AMAT = 100ns \cdot 0.9 + 10ms \cdot 0.1 = 1.00009 \text{ ms}$
 - $AMAT = 100ns \cdot 0.999 + 10ms \cdot 0.001 = 10.0999 \text{ } \mu s$

理想的换页策略 (OPT策略)

假设物理内存中可以存放三个物理页，初始为空，
某应用程序一共需要访问物理页面 1 ~ 5

OPT：优先换出未来最长时间不会再访问的页面

物理页面访问顺序	3	2	3	1	4	3	5	4	2	3	4	3
物理内存中 存放的物理页面												
缺页异常 (共 6 次)	是	是	否	是	是	否	是	否	否	是	否	否

FIFO策略

操作系统维护一个队列用于记录换入内存的物理页号，
每换入一个物理页就把其页号加到队尾，
因此最先换进的物理页号总是处于队头位置

物理页面访问顺序	3	2	3	1	4	3	5	4	2	3	4	3
(该行为队列头) 存储物理页号 的 FIFO 队列												
缺页异常 (共 9 次)	是	是	否	是	是	是	是	否	是	否	是	是

Belady's Anomaly

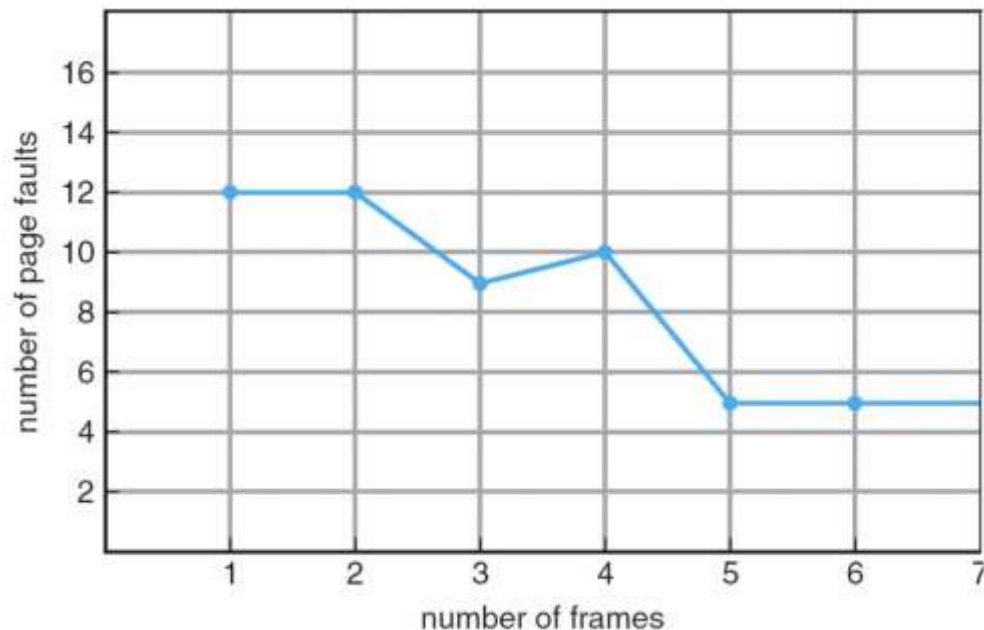
- 访问顺序: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

– 3个物理页: 几次缺页异常?

– 4个物理页: 几次缺页异常?

9次

10次



Second Chance策略

FIFO 策略的一种改进版本：为每一个物理页号维护一个访问标志位。
如果访问的页面号已经处在队列中，则置上其访问标志位。
换页时查看队头：1) 无标志则换出；2) 有标志则去除并放入队尾，继续寻找

物理页面访问顺序	3	2	3	1	4	3	5	4	2	3	4	3
该行是队列头部 FIFO 队列 存储物理页号												
缺页异常（共 6 次）	是	是	否	是	是	否	是	否	是	否	否	否

LRU策略

OS维护一个链表，在每次内存访问后，OS把刚刚访问的内存页调整到链表尾端；每次都选择换出位于链表头部的页面

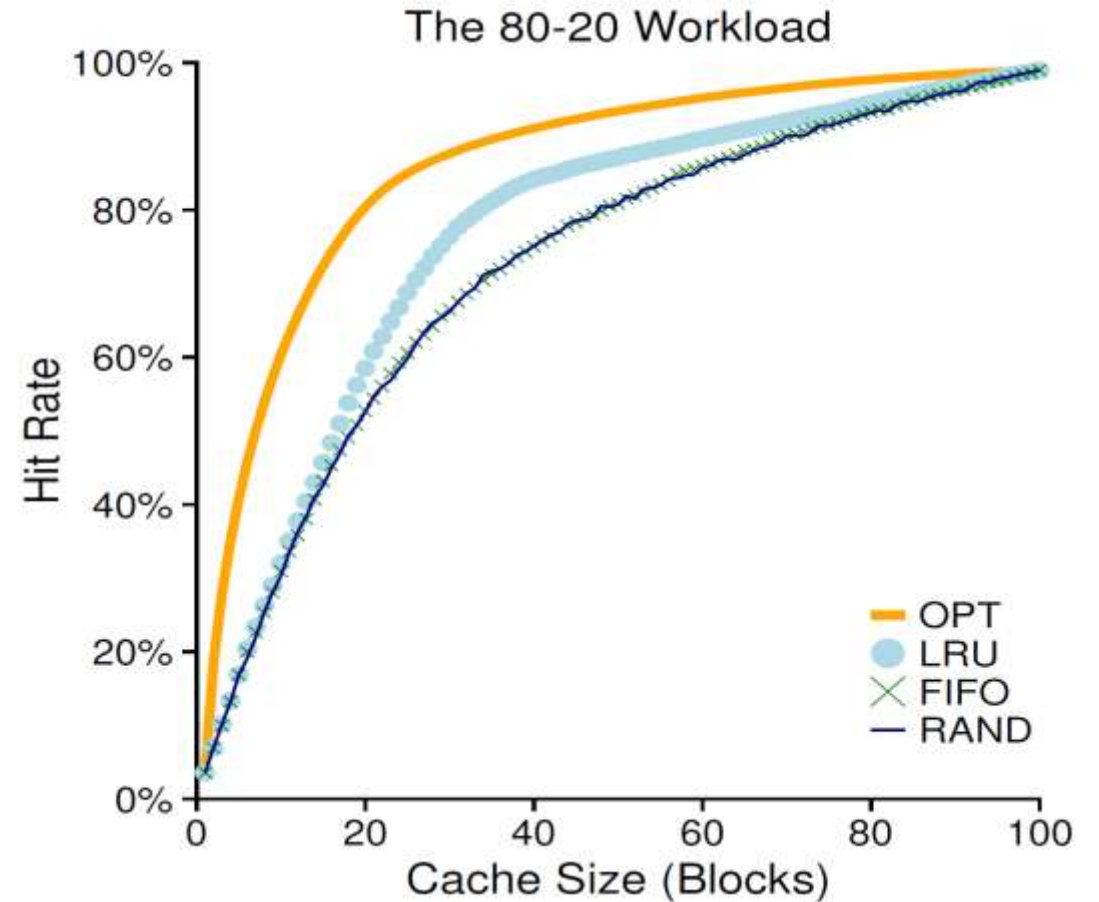
缺点-1：对于特定的序列，效果可能非常差，如循环访问内存

缺点-2：需要排序的内存页可能非常多，导致很高的额外负载

物理页面访问顺序	3	2	3	1	4	3	5	4	2	3	4	3
该行为链表头部 越不常访问的页号 离头部更近												
缺页异常（共7次）	是	是	否	是	是	否	是	否	是	是	否	否

“80-20” Workload

- **Hot pages**
 - 80% of the references
 - is about to 20% of the pages
- **Cold pages**
 - the remaining 20% of the references
 - is about the remaining 80% of the pages

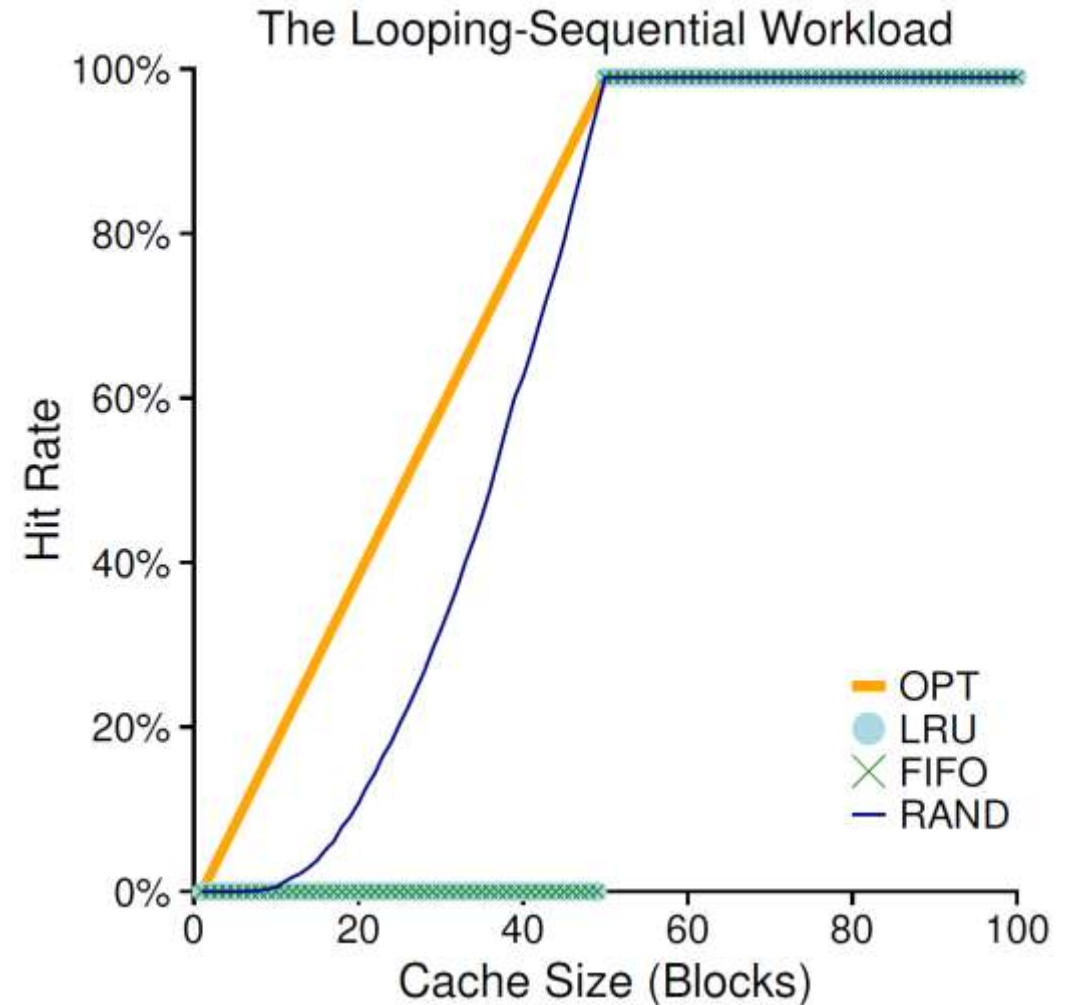


“80-20” Workload

- **Both random and FIFO do reasonably well**
- **LRU does better**
 - it is more likely to hold onto the hot pages
 - those pages have been referred to frequently in the past
 - they are likely to be referred to again in the near future
- **Optimal does best**
 - showing that LRU’s historical information is not perfect
- **Does LRU always do better than random and FIFO?**
 - It depends

“Looping Sequential” Workload

- **Refer to 50 pages in sequence**
 - start at 0, then 1, ..., up to page 49,
 - repeat those accesses
 - for a total of 10,000 accesses to 50 unique pages



“Looping Sequential” Workload

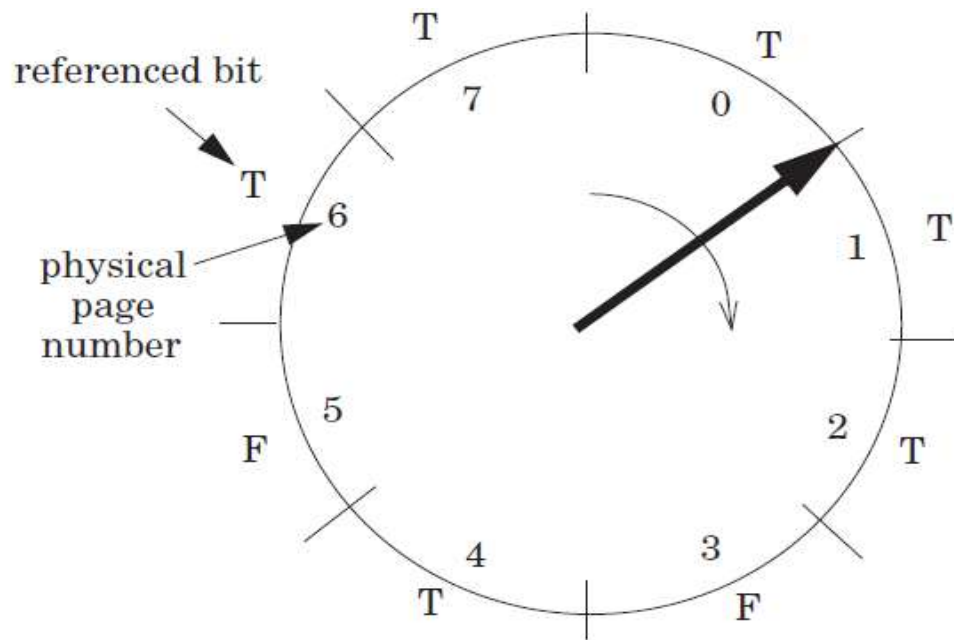
- Common in many applications such as databases
- A worst case for both LRU and FIFO
 - kick out older pages
 - these older pages are going to be accessed sooner than the pages that the policies prefer to keep in cache
 - even with a cache of size 49, hit rate remains 0%
- **Random** fares notably better
 - not quite approaching optimal, but at least achieving a non-zero hit rate
 - not having weird corner-case behaviors
 - **used as TLB replacement policy**

How to implement LRU?

- **Hardware should put a time stamp on each physical page whenever it is accessed**
- **When evicting, it should sort the pages by their time stamps**
- **It may be too many pages to be sorted**
 - One million pages for 4GB memory with 4KB page-size

时钟算法策略

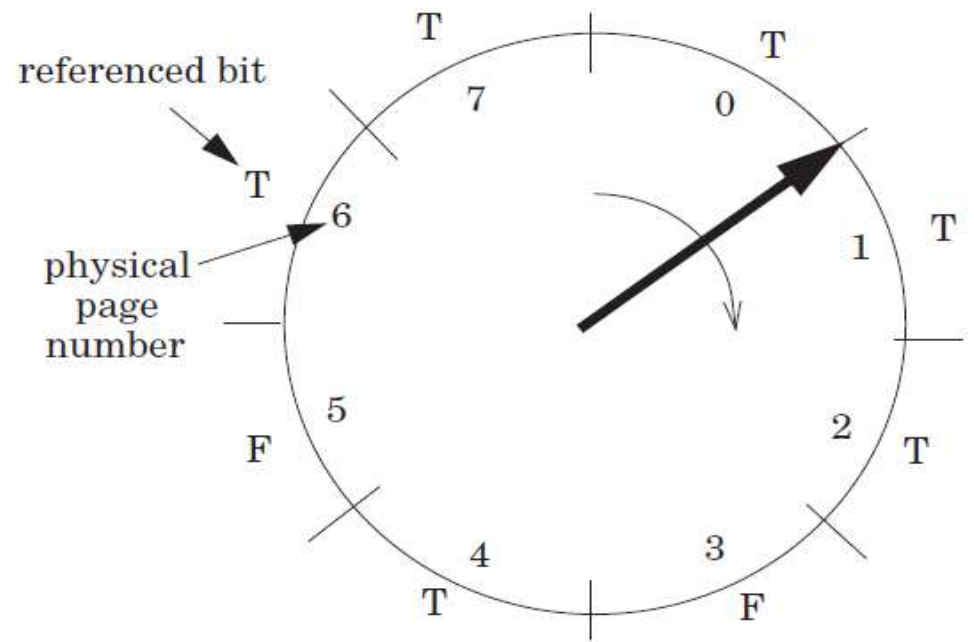
- 精准的LRU策略难以实现
- 物理页环形排列（类似时钟）
 - 为每个物理页维护一个访问位
 - 当物理页被访问时，把访问位设成T
 - OS依次（如顺时针）查看每个页的“访问位”
 - 如果是T，则置成F
 - 如果是F，则驱逐该页



时钟算法策略

针臂：

- 指向换入位置
- 需要换出时：从下一个位置开始转



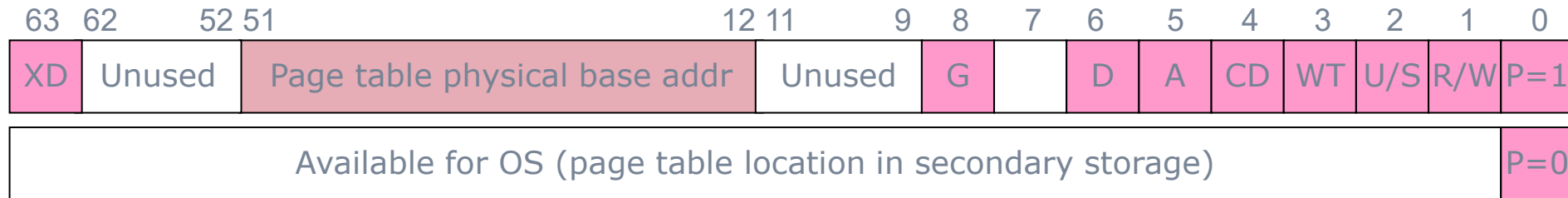
物理页面访问顺序	3	2	3	1	4	3	5	4	2	3	4	3
	3*	3*	3*	3*	4*	4*	4*	4*	2*	2*	2*	2*
		2*	2*	2*	2	3*	3*	3*	3	3*	3	3*
				1*	1	1	5*	5*	5	5	4*	4*
缺页异常	是	是	否	是	是	是	是	否	是	否	是	否

实现时钟算法

```
1 // 在 physical_page 结构体中新增成员变量
2 struct physical_page {
3     ...
4
5     // 记录该物理页被映射到哪些页表项（称为反向映射）
6     list pgtbl_entries;
7 };
```

- 每个**物理页**需要有一个“访问位”
 - MMU在页表项里面为**虚拟页**打上“访问位”
 - 回顾：页表项中的Access Flag
- 如何实现
 - OS在描述物理页的结构体里面记录页表项位置
 - 当物理页被填写到某张页表中时，把页表项的位置记录在元数据中（在Linux中称为“反向映射”：**reverse mapping**）
 - 根据物理页对应的页表项中的“访问位”判断是否驱逐
 - 驱逐某页时应该清空其对应的所有页表项（例如共享内存）

Level 4 Page Table Entry



XD	Disable or enable instruction fetches
Base addr	40 most significant bits of base address of child page table (forces page tables to be 4KB aligned)
G	global page (don't evict from TLB on task switch)
D	Dirty bit (Set by MMU on writes, cleared by software)
A	Reference bit (set by MMU on reads and writes, cleared by software)
CD	Cache disabled(1) or enabled(0) for child page table
WT	Write-through or write-back cache policy
U/S	User or supervisor(kernel) mode access permission
R/W	Read-only or read-write access permission
P	Child page table present in memory(1) or not(0)

Questions

- How does the clock algorithm get the reference bit for Physical Pages?
 - MMU only sets the access bit in VP **Reverse mapping!**
- Can OS evict page table pages?
 - First-level ~ Fourth-level **Feasible except for the first-level.**
- Can OS evict its own pages? **It depends.
But usually not.**

Questions

- How many page fault exceptions may happen when executing one mov instruction? How about two consecutive mov instructions?
- Is that possible to access address 0 legally?
- What is the whole procedure of segmentation faults? (Application, CPU, and OS)